

1. Abstract & Introduction

Large Language Models (LLMs) frequently struggle with multi-step reasoning, particularly in quantitative fields. While Outcome-Supervised Reward Models (ORMs) verify only final answers, Process Reward Models (PRMs) evaluate intermediate reasoning steps, significantly boosting test-time performance through search strategies like Best-of- N (BoN) sampling. Recent breakthroughs like *Math-Shepherd* (2024) successfully trained automated math PRMs without human annotations.

However, constructing domain-specific PRMs (such as *Fin-PRM*) requires massive specialized computational pipelines and datasets. This project explores a zero-training, compute-efficient alternative:

Can a Process Reward Model trained exclusively on pure mathematics generalize effectively to verify deterministic financial math reasoning, or does specialized financial jargon cause verification collapse?

2. Research Question

- **Primary RQ:** To what extent does a mathematical Process Reward Model (e.g., a *Math-Shepherd* derivative) retain verification accuracy when applied to multi-step financial arithmetic problems compared to standard Outcome Reward Models (ORMs)?
- **Secondary RQ:** What are the primary failure modes of a pure-math PRM when scoring financial text (e.g., does it penalize valid mathematical steps due to unknown domain terminology like "EBITDA" or "Revenue growth")?

3. Methodology & Computational Pipeline

Given the strict 1-month timeline and zero-budget resource constraints, **this project involves zero training and relies entirely on open-source local inference.**

[Problem Input] → [Generator LLM (CoT)] → [Generate N Trajectories]

↓
[Select Best Pathway] ← [Evaluate & Score Steps] ← [Math-Shepherd PRM]

1. **Dataset Selection:** We will use **FinQA** (or a subset of 200–500 complex problems), a benchmark for financial analysis over balance sheets requiring numerical reasoning.
2. **Trajectory Generation:** We will use a lightweight local open-weights model (e.g., [Llama-3.2-3B-Instruct](#) or [Qwen-2.5-3B-Instruct](#)) to generate N ($N=8$ or 16) candidate reasoning paths via Chain-of-Thought (CoT) prompting for each question.
3. **Step-Level Verification (The Core Experiment):** * Each trajectory will be parsed into individual sentences/steps.
 - We will employ an open-source Math-PRM checkpoint (e.g., [trl-lib/Qwen2-0.5B-Reward-Math-Shepherd](#) or a similar highly ranked Hugging Face checkpoint).

- The PRM will output a step-by-step success probability for each path.
- 4. **Decoding Selection:** We will implement **Best-of- N (BoN)** decoding, selecting the trajectory with the highest product (or minimum) of step-level scores.
- 5. **Baselines for Comparison:**
 - **Baseline 1:** Majority Voting / Self-Consistency (selecting the most common final answer among the N generated paths).
 - **Baseline 2:** Standard Zero-shot Chain-of-Thought (picking the first generated path).

4. Evaluation Metrics & Error Analysis

- **Quantitative Metric:** Final execution accuracy on the FinQA dataset (comparing PRM selection vs. Majority Voting vs. Greedy Decoding).
- **Qualitative Error Analysis (Critical Component):** I will manually sample 50 cases where the Math-PRM failed or selected a mathematically incorrect path. I will categorize failures into:
 - *Semantic Noise:* PRM penalizes correct math due to financial wording.
 - *Calculation Blindness:* PRM rewards a step that has plausible wording but a subtle math typo.
 - *Premise Failure:* PRM fails to notice that the model extracted the wrong initial number from the financial table.

5. Feasibility Study & Resource Requirements

- **Compute:** All inference will be executed locally or via free compute tiers (e.g., Google Colab T4 GPU or local laptop using [Ollama/HuggingFace](#) transformers). Using highly optimized small models (3B for generation, 0.5B for verification) ensures that experiments can be run in hours, not days.
- **Timeline (4 Weeks):**
 - **Week 1:** Data parsing (FinQA) and code setup for multi-path text generation.
 - **Week 2:** Running the Math-PRM over generated steps; implementing the BoN algorithm.
 - **Week 3:** Metric calculation, baseline comparisons, and rigorous qualitative error analysis.
 - **Week 4:** Report writing and synthesis of conclusions.

6. Expected Contributions

This project provides an empirical boundary analysis of mathematical verifiers. By demonstrating exactly where a pure math PRM succeeds or breaks down in specialized financial documents, it will inform researchers whether cross-domain transfer learning is viable for test-time compute scaling, or if costly domain-specific PRM retraining (like *Fin-PRM*) remains strictly mandatory.